

Manual Maestro: Creación e Integración de Modelos de IA Visual

Este documento proporciona los recursos necesarios para descargar modelos de vanguardia, las instrucciones para crear modelos de imagen desde cero y la metodología para integrarlos con cualquier Modelo de Lenguaje (LLM) como orquestador.

1. Enlaces de Descarga de Modelos SOTA (State-of-the-Art)

A continuación se presentan los enlaces oficiales a los pesos de los modelos más potentes disponibles actualmente para su ejecución local.

Modelo	Versión	Enlace de Descarga (Hugging Face)	Uso Principal
Flux.1 [dev]	12B DiT	Descargar Flux.1-dev	Calidad profesional y texto
Flux.1 [schnell]	12B Distilled	Descargar Flux.1-schnell	Generación rápida (1-4 pasos)
SD 3.5 Large	8.1B MM-DiT	Descargar SD3.5-Large	Realismo y composición
SD 3.5 Medium	2.5B MM-DiT	Descargar SD3.5-Medium	Uso en GPUs de consumo (8GB)
YOLOv12	Vision Model	Descargar YOLOv12	Visión y detección de objetos
Wan2.1	Video Model	Descargar Wan2.1	Generación de video local

2. Manual de Creación de Modelos de Imagen desde Cero

Crear un modelo generativo de imagen implica tres fases fundamentales: preparación de datos, arquitectura y entrenamiento.

Fase A: Preparación del Dataset

1. **Recolección:** Mínimo 100,000 imágenes para un modelo base, o 20-50 para una personalización (LoRA).
2. **Etiquetado (Captioning):** Es crucial usar un LLM de visión (como LLaVA o CogVLM) para describir cada imagen.
 - *Ejemplo de etiqueta:* “Una fotografía de estilo cinematográfico de un astronauta caminando sobre una superficie de cristal bajo un cielo púrpura, 8k, altamente detallado.”
3. **Limpieza:** Eliminar imágenes con marcas de agua, baja resolución o contenido irrelevante.

Fase B: Arquitectura Técnica

Para un modelo moderno, la arquitectura estándar es el **Diffusion Transformer (DiT)**:

- **VAE (Variational Autoencoder):** Entrena un VAE para comprimir imágenes de 1024x1024 a un espacio latente de 128x128. Esto reduce la carga computacional en un 64x.
- **Bloques Transformer:** Utiliza capas de atención (Self-Attention) para que los píxeles “entiendan” su relación con otros píxeles lejanos en la imagen.
- **Text Encoder:** Integra un modelo como **T5-XXL** o **CLIP-L** para procesar los prompts.

Fase C: Entrenamiento

1. **Flow Matching:** Implementa la técnica de Flow Matching para que el modelo aprenda a transformar ruido en imagen siguiendo trayectorias rectas.
2. **Optimización:** Usa optimizadores como **AdamW** y técnicas de precisión mixta (**BF16**) para ahorrar memoria.
3. **Hardware:** Mínimo una GPU H100 o A100 para entrenamiento base. Para LoRAs, una RTX 3090/4090 es suficiente.

3. Integración y Orquestación con LLMs

Cualquier LLM (Llama 3, GPT-4, Claude) puede actuar como el “cerebro” que controla estos modelos de imagen.

El Rol del LLM Orquestador

El LLM no genera la imagen, sino que realiza tres tareas críticas:

1. **Expansión de Prompts:** Convierte una idea simple del usuario en un prompt técnico detallado.
2. **Gestión de Expertos:** Decide si usar el modelo de “Rostros”, el de “Paisajes” o el de “Arquitectura” basándose en la intención.
3. **Validación de Visión:** Recibe la salida de **YOLOv12** y decide si la imagen es correcta o debe regenerarse.

Ejemplo de Código (Python + Diffusers)

```
import torch
from diffusers import FluxPipeline
from transformers import AutoModelForCausalLM, AutoTokenizer

# 1. Cargar el LLM Orquestador (Llama-3)
llm_name = "meta-llama/Llama-3.2-3B-Instruct"
tokenizer = AutoTokenizer.from_pretrained(llm_name)
llm = AutoModelForCausalLM.from_pretrained(llm_name, torch_dtype=torch.bf16)

# 2. Cargar el Modelo de Imagen (Flux)
pipe = FluxPipeline.from_pretrained("black-forest-labs/FLUX.1-dev", torch_dtype=torch.bf16)
pipe.enable_model_cpu_offload()

def generar_con_orquestador(user_input):
    # El LLM expande el prompt
    prompt_sistema = f"Convierte esta idea en un prompt detallado para IA generativa de imágenes. (Lógica de generación de texto del LLM aquí...)"
    prompt_expandido = f"{user_input} Una obra maestra digital de..."

    # El modelo de imagen genera el resultado
    image = pipe(prompt_expandido, height=1024, width=1024, guidance_scale=3.5).images[0]
    return image
```

4. Guía de Puesta en Funcionamiento Local

1. **Instalación de Dependencias:**

```
pip install torch torchvision torchaudio --index-url https://download.
pip install diffusers transformers accelerate sentencepiece ultralytic
```

2. **Gestión de VRAM:** Si tienes menos de 24GB de VRAM, usa la técnica de **Quantization** (GGUF o bitsandbytes) para cargar los modelos en 4 u 8 bits.
3. **Interfaz Recomendada:** Descarga [ComfyUI](#) para una gestión visual de los modelos y la orquestación mediante nodos.